

Informe del Proyecto Final

Autor: Jesús Redondo

Proyecto: Catálogo Mayorista, panel de inventario dirigido por datos

[Aplicación desplegada](#)

Fecha de entrega: 30 de agosto de 2026

1. La aplicación

Catálogo Mayorista es un panel de inventario para un distribuidor mayorista. Toma un archivo de datos con 240 productos y construye a partir de él dos pantallas: un panel con los indicadores del negocio y la cola de reposición, y un catálogo con búsqueda, filtros y vistas guardadas.

Está construido con Vite, React 19, TypeScript y Tailwind CSS v4.

2. Herramientas utilizadas

El desarrollo se realizó con OpenCode en sus modos Plan y Build, conectado a los proveedores de modelo de Google AI Studio y OpenRouter bajo el esquema BYOK. El diseño de la interfaz se hizo con Google Stitch y la publicación en Vercel.

Sobre la herramienta de diseño

La asignación contempla Open Design para la fase de diseño y admiten alternativas siempre que se declaren en el informe. En este proyecto se utilizó Google Stitch.

Stitch produjo un sistema de diseño al que llamó Industrial Inventory Ledger: una paleta construida sobre un verde de bodega, fondo de papel cálido y las tipografías IBM Plex Sans e IBM Plex Mono. El prompt completo y las pantallas exportadas quedaron guardados en la documentación del proyecto.

No todo lo que devolvió se adoptó. Su panel de detalle proponía nueve campos que el archivo de datos no contenía: peso, volumen, material, origen, plazo de entrega, garantía, código de barras, arancel y marca de sincronización. En lugar de simularlos en la vista, se amplió el archivo origen para que esos campos existieran de verdad.

3. Cumplimiento de las normativas

Norma 1. Cero código manual

Ninguna función, componente o integración se escribió a mano. Cada pieza procede de una instrucción al agente, trabajando en los modos Plan y Build: primero se pedía el plan de arquitectura y se aprobaba, y solo entonces se pasaba a construir.

Las instrucciones están transcritas en el registro de prompts, y el historial de 25 commits conserva el orden en que se generaron.

Norma 2. Integración de contexto de datos

El archivo base del proyecto contiene 240 registros repartidos en 8 categorías, con 24 campos por producto. Solo tres están siempre presentes, que son el identificador, el nombre y la categoría. El resto puede venir vacío, y una de las claves, la del almacén, puede directamente no existir en el registro.

El archivo se inyectó como contexto de dos formas complementarias. La primera, como especificación para el agente: el contrato completo está transcrito en la skill del proyecto, que el agente carga antes de tocar cualquier vista, y de ahí salieron los tipos de TypeScript y las reglas de tolerancia a valores ausentes. La segunda, como fuente en tiempo de ejecución: la aplicación descarga el archivo y deriva de él las categorías con su conteo, los rangos y los umbrales del semáforo de inventario.

No hay una sola categoría, marca, proveedor ni rango de precios escrito en el código. Sustituir el archivo por otro con el mismo contrato cambia la aplicación entera sin tocar una línea.

Norma 3. Skill y comandos personalizados

La skill “catalogo-mayorista” es la especificación que el agente carga automáticamente al tocar cualquier vista. Fija el contrato del archivo de datos, una tabla de tolerancia a valores ausentes caso por caso, los tokens de diseño y una lista de prohibiciones explícitas, entre ellas no usar asertos de tipo para silenciar al compilador ni inventar datos dentro de los componentes.

El comando “renderizar_tarjetas” orquesta la construcción de la lista de productos en cinco pasos encadenados: leer el contrato recorriendo el archivo real en lugar de asumirlo, derivar las opciones desde los datos, resolver los casos de campos ausentes, construir la vista con su estado vacío y, por último, verificar. Ese quinto paso es el que lo distingue de una petición suelta, porque el agente no puede declararse terminado sin ejecutar la comprobación de tipos, la compilación y el repaso de valores nulos.

El comando “auditar_datos” delega en el agente auditor, contrasta su reporte contra el código y registra el resultado.

Norma 4. Agente personalizado

El agente “auditor-datos” tiene un alcance deliberadamente cerrado. Se declara como subagente y con permisos de edición y escritura denegados, de manera que la separación de responsabilidades es una restricción del entorno y no una recomendación escrita en prosa. Puede leer el archivo y contar sobre él, pero no puede tocar el código.

Su trabajo consiste en censar las claves del archivo, cuantificar anomalías como claves ausentes, tipos mixtos, rangos inválidos e incoherencias internas, y traducir cada una

en un requisito verificable para la interfaz. Su instrucción central evita el fallo más común de un agente auditor:

Todo número que reportes debe venir de un conteo real sobre el archivo.
Nunca reportes una cifra que no calculaste.

El flujo entre agentes se verificó en ejecución. Al lanzar el comando auditar_datos sobre el campo de existencias, el comando desplegó sus cuatro pasos y delegó el primero en el subagente, que a su vez ejecutó lo siguiente:

Delegar auditoría de stock al subagente @auditor-datos
Auditor-Datos Task, auditoría de stock en productos.json
Bash node auditar.mjs

La última línea es la relevante. El subagente no estimó las cifras: escribió un script desechable y contó sobre el archivo, que es exactamente lo que su instrucción le exige.

Norma 5. Refactorización y depuración autónoma

Ningún error se corrigió a mano. El ciclo fue siempre el mismo: capturar el error tal como lo emitió la herramienta, pedir al agente la causa raíz antes que el parche, aplicar su corrección y volver a verificar. Los siete casos están documentados en la bitácora del proyecto. Dos de ellos ilustran por qué ese orden importa.

El primero fue un error de tipos en la gráfica. La compilación falló y la solución rápida, que era anotar el parámetro como tipo libre, habría compilado al primer intento. Pedir la causa antes que el parche reveló que el error describía un caso real, porque la librería puede invocar ese callback con un valor ausente, y silenciarlo reintroducía el riesgo de operaciones inválidas que toda la interfaz está construida para evitar.

El segundo fue un descuento aplicado sobre un precio inexistente. Tres productos mostraban un porcentaje de rebaja junto a la etiqueta de precio ausente. El dato era legítimo, porque el precio y la regla de descuento son dos altas distintas del sistema de origen. Lo incorrecto era la vista, que hacía pasar por sensata una combinación que no lo es. El porcentaje dejó de dibujarse y la incoherencia pasó a reportarse aparte, en el panel de calidad del catálogo.

Sobre los valores nulos, la interfaz distingue tres situaciones que no son lo mismo: el dato que falta por capturar, el que no aplica a ese artículo y el que vale cero. Colapsarlas en una sola etiqueta haría mentir a la interfaz. Por ejemplo, unas existencias sin registrar se muestran como sin dato y nunca como agotado; una garantía nula se muestra como no aplica y no se atenúa; y una fecha de alta ausente muestra un guion, sin construir nunca un objeto de fecha a partir de un valor nulo, que en JavaScript devolvería el 31 de diciembre de 1969 como si fuera un dato real.

Estas reglas quedaron fijadas como 31 pruebas automáticas que montan la aplicación con el archivo real y comprueban que ningún valor inválido llega a la pantalla.

Norma 6. Despliegue a producción

La aplicación está publicada en Vercel y es accesible de forma pública en la dirección <https://proyecto-opencode.vercel.app>

Antes de cada publicación se ejecuta una verificación que corre la comprobación de tipos, el análisis estático, las pruebas y la compilación.

4. Prompts utilizados

La transcripción completa está en el registro de prompts del proyecto. Se agrupan en tres tipos.

Los conversacionales son los escritos directamente en el chat. Fueron pocos y de alto nivel. El primero pedía leer las directrices y explicar los entregables, y el segundo encargaba la construcción con la instrucción de buscar el proyecto más simple que cumpliera las seis normas.

Los estructurados son los guardados como archivos de configuración del agente. Ahí vive el grueso de la orquestación, y la diferencia práctica es considerable: un prompt escrito en el chat se ejecuta una vez y se pierde, mientras que uno guardado como comando se ejecuta igual la décima vez, incluso en otra máquina.

Los de depuración siguieron siempre el mismo patrón ante cada fallo:

Este es el error exacto que devolvió la compilación. Antes de proponer un parche, explícame la causa raíz: por qué el compilador considera esto un error y qué caso real está describiendo. Después aplica la corrección y vuelve a ejecutar la verificación.

La segunda mitad de ese prompt es la que importa. Sin ella, el agente entrega el parche que hace desaparecer el mensaje, que no siempre es el mismo que arregla el problema.

Documentos de apoyo

El proyecto incluye el registro de prompts utilizados, la bitácora con los siete fallos y su causa raíz, el prompt de diseño entregado a Google Stitch junto con el análisis de su resultado, las pantallas exportadas de esa herramienta y los archivos de configuración del agente, que contienen la skill, los dos comandos y el subagente auditor.